

EX.NO: 3

SHELL PROGRAMMING BASICS

AIM

To execute the fundamentals of shell programming such as control flow statements.

EXISTING PROBLEM

- It is not possible to perform more than one task at a time using shell command

SHELL SCRIPTS (MULTITASKING)

- In order to solve the problems of shell command, the shell programming is introduced here
- Doing more than one job at a time (multitasking)
- It is also called as **shell programming**

VARIABLES SECTION

- Names given to the memory location
- Shell program supports **dynamic data typed system** which means that no need to use specific data type for variables declaration

Syntax

Variable-Name=Initial-Value

Example

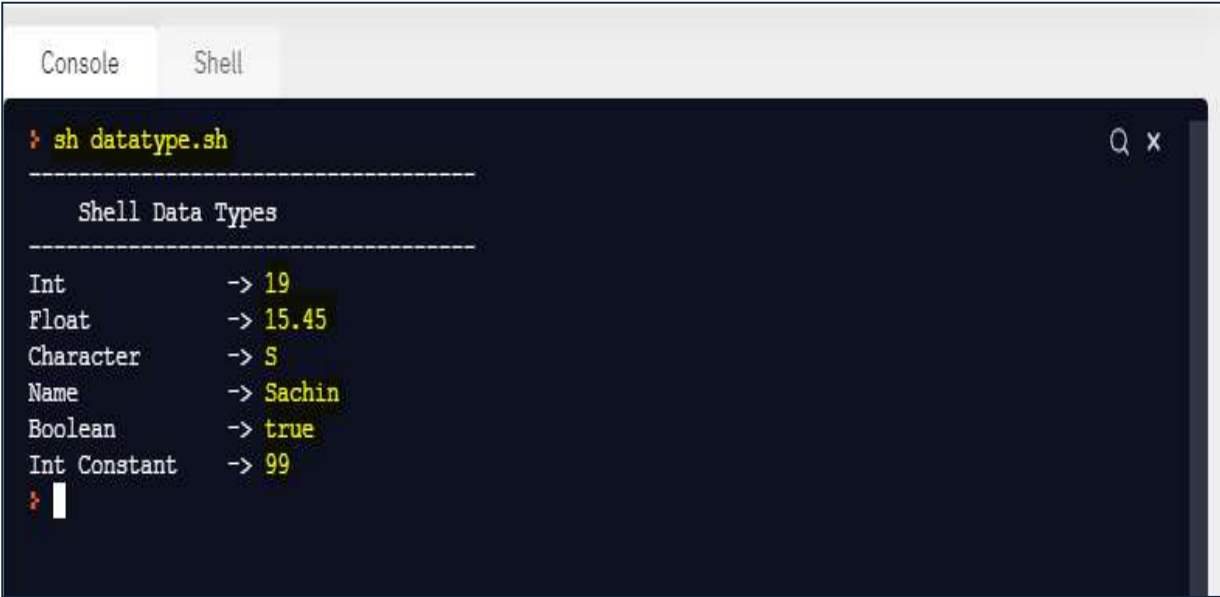
a=10	# integer type
str="Sachin"	# string type
c='G'	# character type
f=true	# boolean value
readonly id=14	# integer constant

I. EXAMPLE OF DATA TYPES IN SHELL CODE

SOURCE CODE

```
echo "-----"
echo "\tShell Data Types"
echo "-----"
# variables definition
a=19
b=15.45
c='S'
str="Sachin"
flag=true
# constant variable definition
readonly id=99
echo "Int\t\t\t\t-> $a"
echo "Float\t\t\t-> $b"
echo "Character\t\t-> $c"
echo "Name\t\t\t\t-> $str"
echo "Boolean\t\t\t-> $flag"
echo "Int Constant\t-> $id"
```

2. OUTPUT



```
sh datatype.sh

-----
Shell Data Types
-----
Int          -> 19
Float        -> 15.45
Character     -> S
Name         -> Sachin
Boolean      -> true
Int Constant -> 99
$
```

COMMENT LINE STATEMENTS

- Usually these statements are ignored by compiler or interpreter
- Like c/c++, shell supports two types of comment line statements. They are
 1. Single line statement
 2. Multi line statements

1. Single line statement

- The single line statement is indicated by **# symbol** in shell program

Example

```
# This is single line statement
```

2. Multi line statements

- It is used to ignore more than one statements
- This is indicated by **:' ' symbol** in shell program

Example

```
:' Variable Declarations  
    a=20  
    k=25  
,
```

SELECTION STATEMENTS

1. Simple If statement
2. If else Statement
3. If else if Statement
4. Case Statement

1. Simple If Statement

Syntax

```
if [ condition ]  
then  
    true statement  
fi
```

- It is an important to note that, the space should be given before and after the **operator symbol** []
- You can use test keyword instead of the operator symbols []

2. If else Statement

Syntax

```
if [ condition ]  
then  
    true statement  
else  
    false statement  
fi
```

3. If...elif...else Statement

Syntax

```
if [ condition ]  
then  
    true statement  
elif [ condition ]  
then  
    true statement  
else  
    false statement  
fi
```

- It is an important to note that, the simple if, if else and if-elif-else should be closed by **fi** keyword.

4. Case Statement

- It is equivalent to switch case statements in c language
- It is used to execute several statements based on the value of expression
- This is done by using the reserved word **case**
- It is an alternative option for if..elif..else statements

Syntax

```
case <variable> in  
Pattern 1)  
    Commands / statements  
    ;;  
Pattern 2)  
    Commands / statements  
    ;;  
    ...  
esac
```

Where,

;; → represents the break part in case statements

LOOPING STATEMENTS

1. While loop (while)
2. Until loop (until)
3. For loop (for)

1. While loop

Syntax

```
while [ condition ]  
do  
    true statement  
done
```

Infinite While loop

- It is an important to note that, **the colon (:) operator or true keyword** is used for creating an infinite loop
- The **colon (:) operator** is used instead of the operator symbols **[]**

II. EXAMPLE OF INFINITE LOOP USING WHILE LOOP

SOURCE CODE

```
while :  
do  
    echo "Hello World"  
done
```

(OR)

```
while true  
do  
    echo "Hello World"
```

done

OUTPUT

[illegible]

2. Until loop

- Here loop is executing until the **condition is false**
- If the **condition becomes true** it will exit from the loop

Syntax

```
until [ condition ]  
do  
    true statement  
done
```

III. EXAMPLE OF UNTIL LOOP

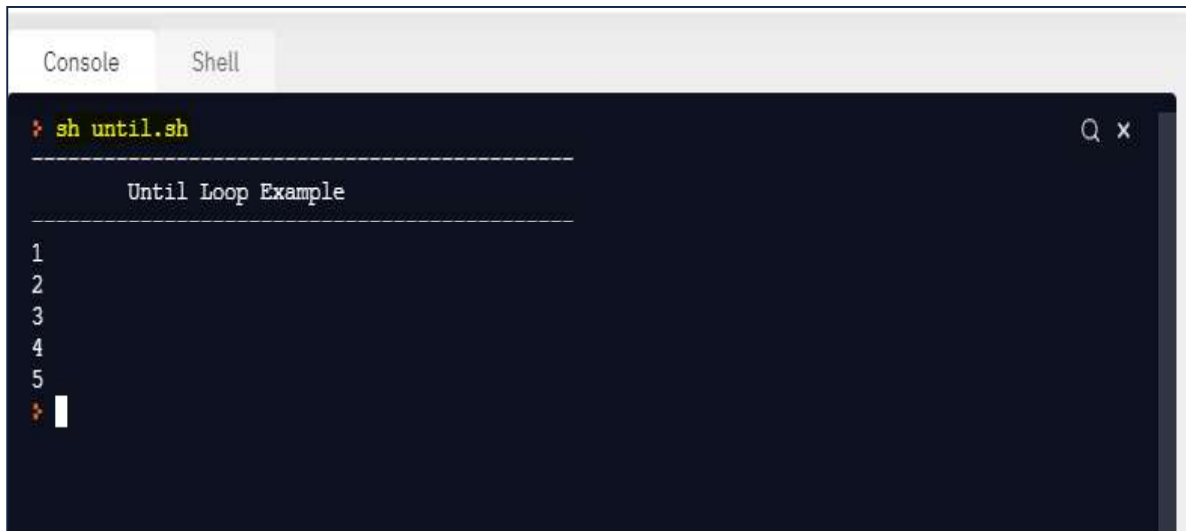
SOURCE CODE

```
echo "-----"  
echo "\t\tUntil Loop Example"  
echo "-----"  
i=1  
until [ $i -gt 5 ]  
do  
    echo $i  
    i=`expr $i + 1`  
done
```

Here the looping statements are executed until the condition becomes fail like **1>5, 2>5, 3>5, 4>5, 5>5**

This loop will terminate whenever the condition becomes true like **6>5**

OUTPUT



```
sh until.sh
-----
Until Loop Example
-----
1
2
3
4
5
```

3. For loop

Syntax

```
for variable in w1 w2 ... wn
do
    true statement
done
```

Where,

Variable can be any user defined name

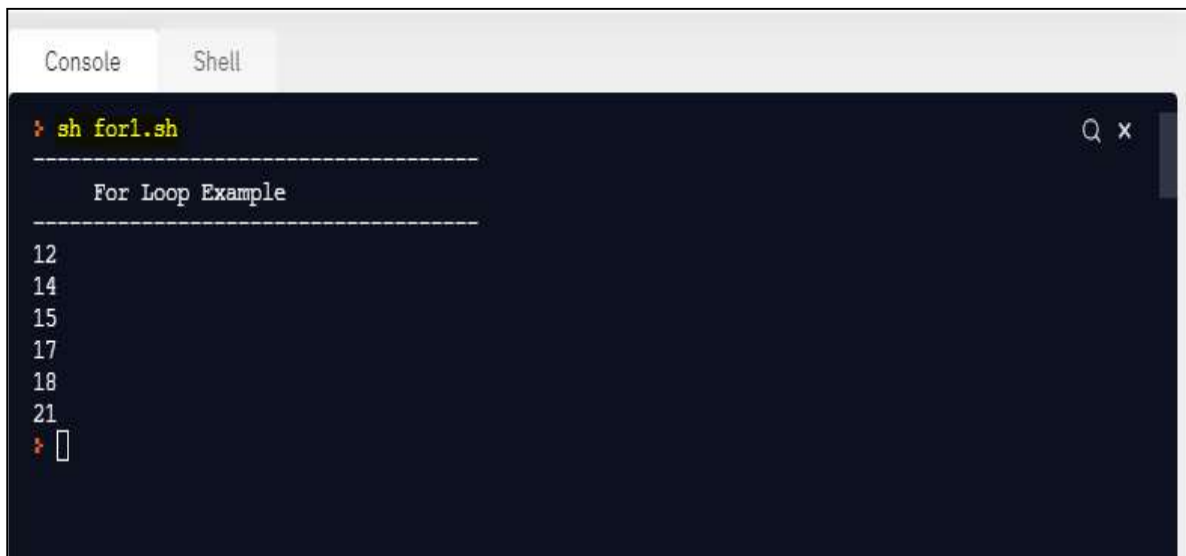
w1 w2 ...wn → list of the values separated by spaces

IV. EXAMPLE OF FOR LOOP

SOURCE CODE

```
echo "-----"  
echo "\t For Loop Example"  
echo "-----"  
for i in 12 14 15 17 18 21  
do  
    echo $i  
done
```

OUTPUT



```
sh for1.sh  
-----  
    For Loop Example  
-----  
12  
14  
15  
17  
18  
21  
$
```

V. CHARACTERS AND STRING RETRIEVAL USING FOR LOOP

SOURCE CODE

```
echo "-----"  
echo "\t Char & String Retrieval using for loop"  
echo "-----"  
for i in 'Sachin' 'B' "C" "D" 'E'  
do  
    echo $i  
done
```

OUTPUT

```
Console  Shell
GNU bash, version 4.4.20(1)-release (x86_64-pc-linux-gnu)
> sh for2.sh
-----
Char & String Retrieval using for loop
-----
Sachin
B
C
D
E
> 
```

VI. DISPLAYING FILES AND DIRECTORIES USING FOR LOOP

SOURCE CODE

```
echo "-----"
```

```
echo "\t Listing Files and Folder using for loop"
```

```
echo "-----"
```

```
# get all the files and store them to variable
```

```
fset=`ls`
```

```
k=1
```

```
# loop the variable fset
```

```
for i in $fset
```

```
do
```

```
    echo "$k. $i"
```

```
    k=`expr $k + 1`
```

```
done
```

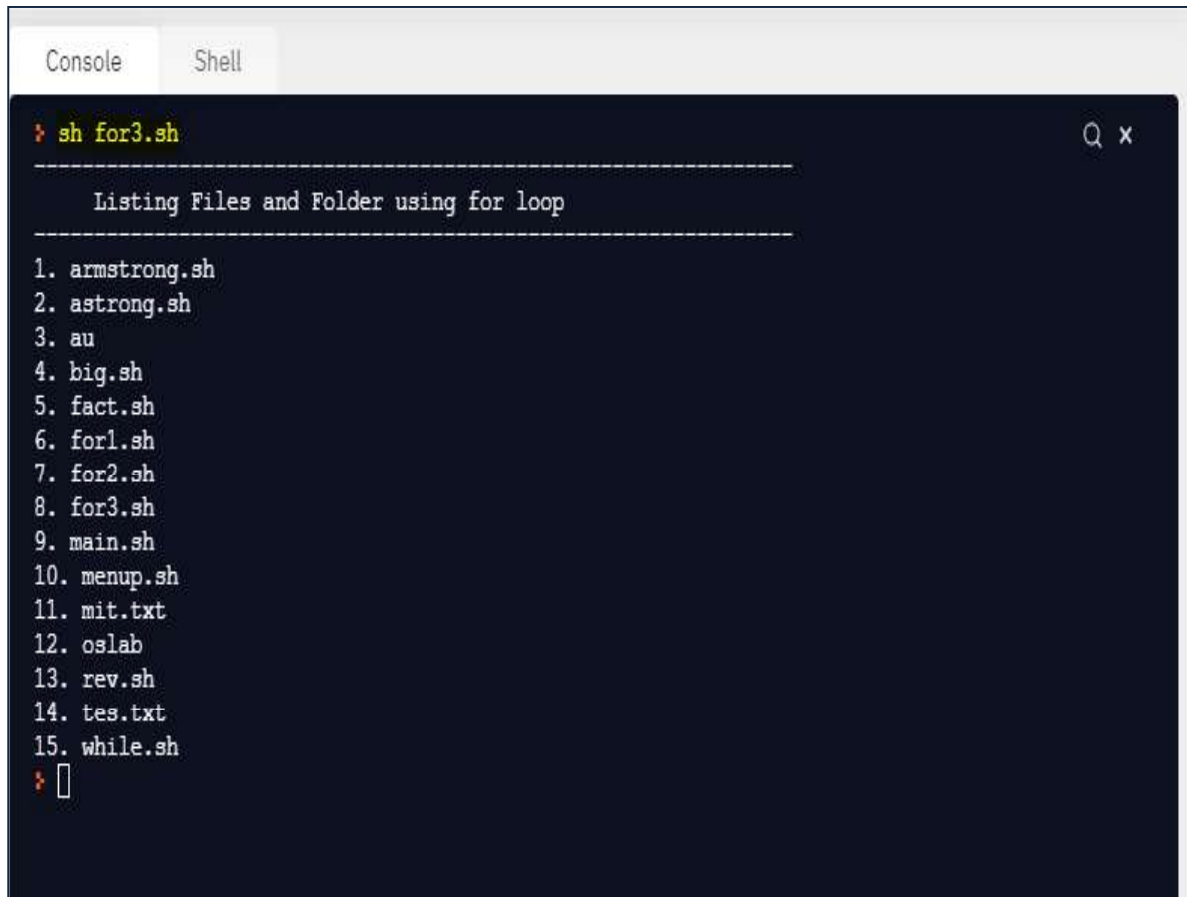
Store list of files to the variable fset using ls command.

k=k+1 or k++

Command Substitution:

Storing the output of command to a user defined variable. This is done by using the operator ``command`` or `$(command)`

OUTPUT



```
sh for3.sh

-----
Listing Files and Folder using for loop
-----
1. armstrong.sh
2. astrong.sh
3. au
4. big.sh
5. fact.sh
6. for1.sh
7. for2.sh
8. for3.sh
9. main.sh
10. menup.sh
11. mit.txt
12. oslab
13. rev.sh
14. tes.txt
15. while.sh
❏
```

VII. FACTORIAL OF A NUMBER

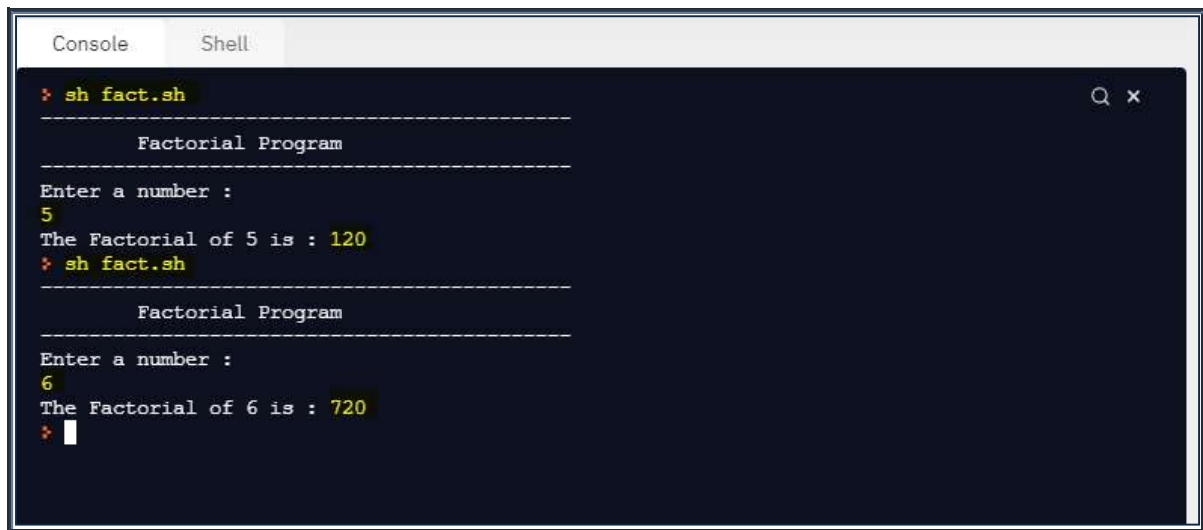
Language	:	shell (.sh)
Editor	:	replit.com (Online Linux Terminal)
OS	:	Windows 10

SOURCE CODE

```
echo "-----"
echo "\t\tFactorial Program"
echo "-----"
echo "Enter a number : "
read n
i=0
f=1
while [ $i -lt $n ]
```

```
do
# increment i by 1
    i=`expr $i + 1`
    f=`expr $f \* $i`
done
echo "The Factorial of $n is : $f"
```

OUTPUT



```
Console Shell
❖ sh fact.sh
-----
Factorial Program
-----
Enter a number :
5
The Factorial of 5 is : 120
❖ sh fact.sh
-----
Factorial Program
-----
Enter a number :
6
The Factorial of 6 is : 720
❖
```

C STYLE CODING IN SHELL

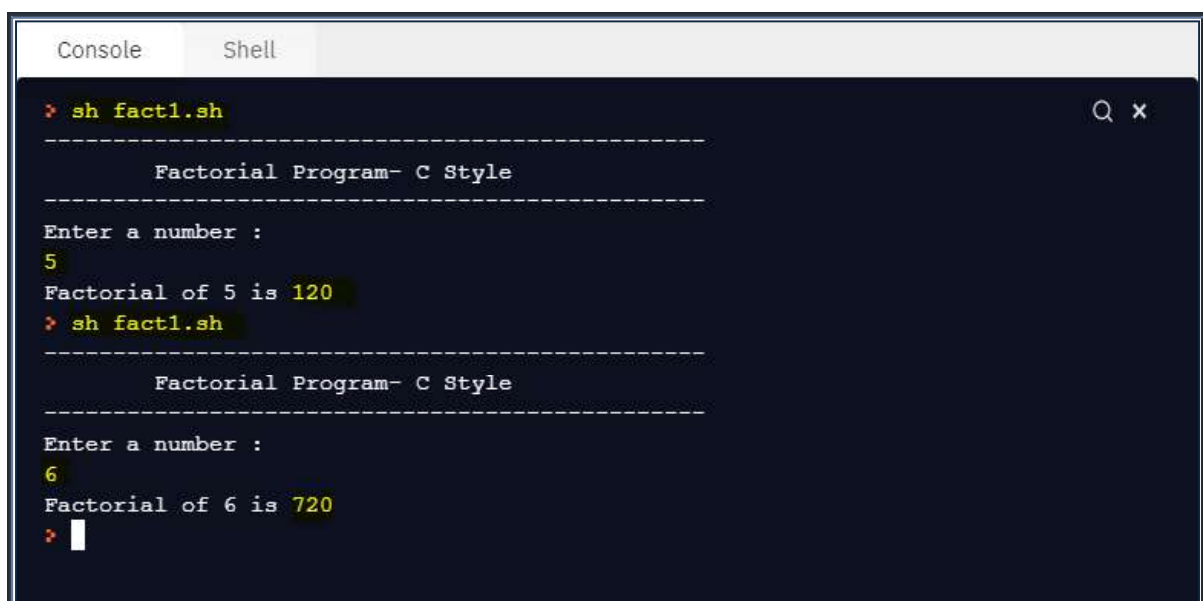
- In shell, we can use c style coding for looping statements and expressions
- The expressions are defined by **\$(exp)**. Here each is also closed by () symbol.

VIII. FACTORIAL OF A NUMBER USING C-STYLE

SOURCE CODE

```
echo "-----"
echo "\t\tFactorial Program- C Style"
echo "-----"
echo "Enter a number : "
read n
i=1
f=1
# while loop
while [ $i -le $n ]
do
# expression 1 in c-style
  f=$((f*i))
# expression 2 in c-style
  i=$((i+1))
done
echo "Factorial of $n is $f"
```

OUTPUT



The screenshot shows a terminal window with two tabs: 'Console' and 'Shell'. The 'Console' tab is active. The terminal displays the output of running a script named 'fact1.sh'. The script prints a header 'Factorial Program- C Style' between two dashed lines, prompts for a number, and then outputs the factorial. The first run shows the factorial of 5 is 120. The second run shows the factorial of 6 is 720. The prompt is currently empty.

```
> sh fact1.sh
-----
      Factorial Program- C Style
-----
Enter a number :
5
Factorial of 5 is 120
> sh fact1.sh
-----
      Factorial Program- C Style
-----
Enter a number :
6
Factorial of 6 is 720
> 
```

IX. REVERSE OF A NUMBER

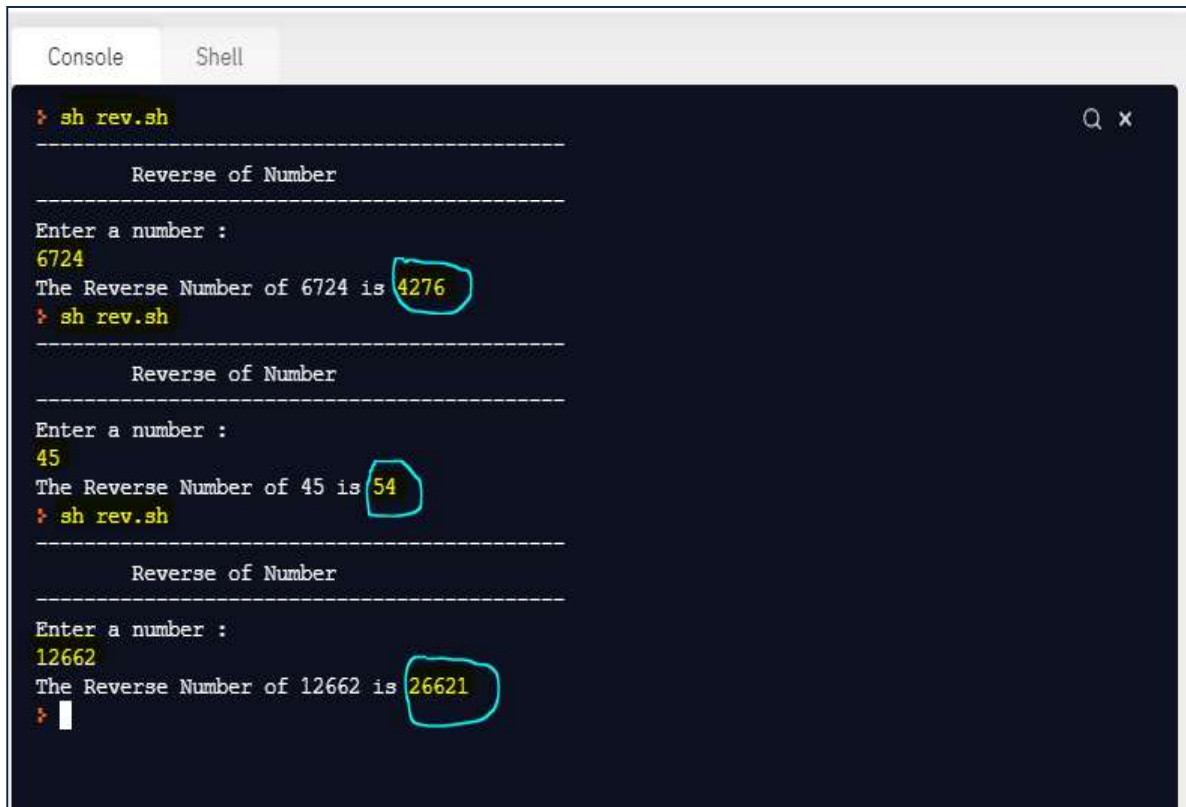
SOURCE CODE

```
echo "-----"
echo "\t\tReverse of Number"
echo "-----"
echo "Enter a number : "
read n
duplicate=$n
res=0
while [ $n -ne 0 ]
do
    # find the remainder
    rem=`expr $n % 10`
    # multiply reverse number with 10
    res=`expr $res \* 10`
    # add the resultant number with remainder number
    res=`expr $res + $rem`
    # divide n by 10
    n=`expr $n / 10`
done
echo "The Reverse Number of $duplicate is $res"
```

The diagram illustrates the logic of the shell script by mapping code lines to descriptive blocks:

- `echo "-----"` and `echo "\t\tReverse of Number"` are connected to the first block.
- `echo "-----"` and `echo "Enter a number : "` are connected to the second block.
- `read n` is connected to the second block.
- `duplicate=$n` and `res=0` are connected to the third block.
- `while [ $n -ne 0 ]` is connected to the third block.
- `do` is connected to the fourth block.
- `# find the remainder` and `rem=`expr $n % 10`` are connected to the fourth block.
- `# multiply reverse number with 10` and `res=`expr $res * 10`` are connected to the fifth block.
- `# add the resultant number with remainder number` and `res=`expr $res + $rem`` are connected to the fifth block.
- `# divide n by 10` and `n=`expr $n / 10`` are connected to the sixth block.
- `done` is connected to the sixth block.
- `echo "The Reverse Number of $duplicate is $res"` is connected to the seventh block.

OUTPUT



```
Console Shell
❖ sh rev.sh
-----
Reverse of Number
-----
Enter a number :
6724
The Reverse Number of 6724 is 4276
❖ sh rev.sh
-----
Reverse of Number
-----
Enter a number :
45
The Reverse Number of 45 is 54
❖ sh rev.sh
-----
Reverse of Number
-----
Enter a number :
12662
The Reverse Number of 12662 is 26621
❖
```

X. ARMSTRONG NUMBER

SOURCE CODE

```
echo "-----"
echo "\t\tArmstrong Number"
echo "-----"
echo "enter a number"
read n
# variables declarations
dup=$n
arm=0
# while loop
while test $n -ne 0
do
# find remainder of a number
rem=`expr $n % 10`
# multiply rem with three times
rem=`expr $rem \* $rem \* $rem`
# add the resultant remainder with arm variable
```

while (n!=0)

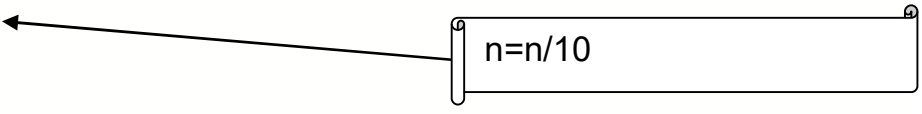
rem=n%10

rem=(rem*rem*rem)

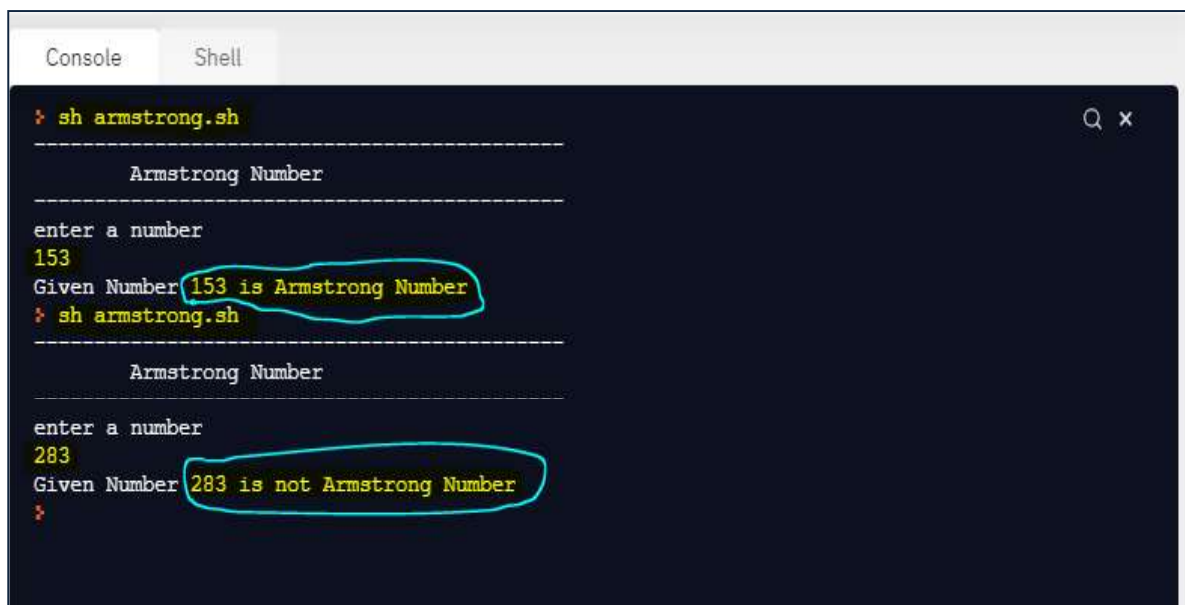

```

    arm=`expr $arm + $rem`
# divide n by 10
    n=`expr $n / 10`
done
if [ $dup -eq $arm ]
then
    echo "Given Number $dup is Armstrong Number"
else
    echo "Given Number $dup is not Armstrong Number"
fi

```



OUTPUT



```

sh armstrong.sh

Armstrong Number
-----
enter a number
153
Given Number 153 is Armstrong Number
sh armstrong.sh

Armstrong Number
-----
enter a number
283
Given Number 283 is not Armstrong Number

```

XI. MENU DRIVEN PROGRAM

SOURCE CODE

```

while [ true ]
do
    echo "-----"
    echo "\t\t Menu Program"
    echo "-----"
    echo "1. View Files \t 2.Date"
    echo "3. Users List \t4.Calendar"
    echo "5. Exit"

```

```
echo "\tEnter ur choice : "
read ch
#switch case
case $ch in
    1) ls;;
    2) date;;
    3) w;;
    4) cal;;
    5) exit;;
esac
# read choice from user for continuation of program execution
echo "Do you want to continue : Press Yes/No"
read ch
if [ $ch = "yes" ] || [ $ch = "Yes" ] || [ $ch = "YES" ]
then
    continue
else
    exit
fi
done
```

NOTE

- It is an important to note that, the single equal operator (=) is used for **string comparison** and the symbol **-eq** or **==** is used for **number comparison** in the shell program.

OUTPUT

```
Console Shell
❏ sh menup.sh
-----
Menu Program
-----
1. View Files    2.Date
3. Users List    4.Calendar
5. Exit
Enter ur choice :
1
armstrong.sh au fact.sh menup.sh oslab tes.txt
astrong.sh big.sh main.sh mit.txt rev.sh
Do you want to continue : Press Yes/No
Yes
-----
Menu Program
-----
1. View Files    2.Date
3. Users List    4.Calendar
5. Exit
Enter ur choice :
3
05:10:41 up 1:47, 0 users, load average: 8.98, 7.29, 6.29
USER      TTY      FROM          LOGIN@   IDLE   JCPU   PCPU WHAT
Do you want to continue : Press Yes/No
yes
-----
Menu Program
-----
1. View Files    2.Date
3. Users List    4.Calendar
5. Exit
Enter ur choice :
4
April 2021
Su Mo Tu We Th Fr Sa
                1  2  3
 4  5  6  7  8  9 10
11 12 13 14 15 16 17
18 19 20 21 22 23 24
25 26 27 28 29 30

Do you want to continue : Press Yes/No
YES
-----
Menu Program
-----
1. View Files    2.Date
3. Users List    4.Calendar
5. Exit
Enter ur choice :
2
Thu Apr 29 05:10:57 UTC 2021
Do you want to continue : Press Yes/No
```

OUTPUT – (CON)

```
Do you want to continue : Press Yes/No
yes
-----
Menu Program
-----
1. View Files    2.Date
3. Users List   4.Calendar
5. Exit
Enter ur choice :
5
█
```

COMMAND LINE ARGUMENTS (POSITIONAL PARAMETERS)

- Process of passing the input arguments to the program at the time of execution is called as command line arguments
- In shell, this is done with help of **\$** symbol

S.N	POSITIONAL PARAMETERS	DESCRIPTION
1.	\$0	Indicates the filename itself
2.	\$1	Indicates the first argument
3.	\$2	Indicates the second argument
	...	
4.	\$*	Represents the total number of input arguments which are submitted to the shell program
5.	\$#	Shows the count of total number of arguments passed to the shell program

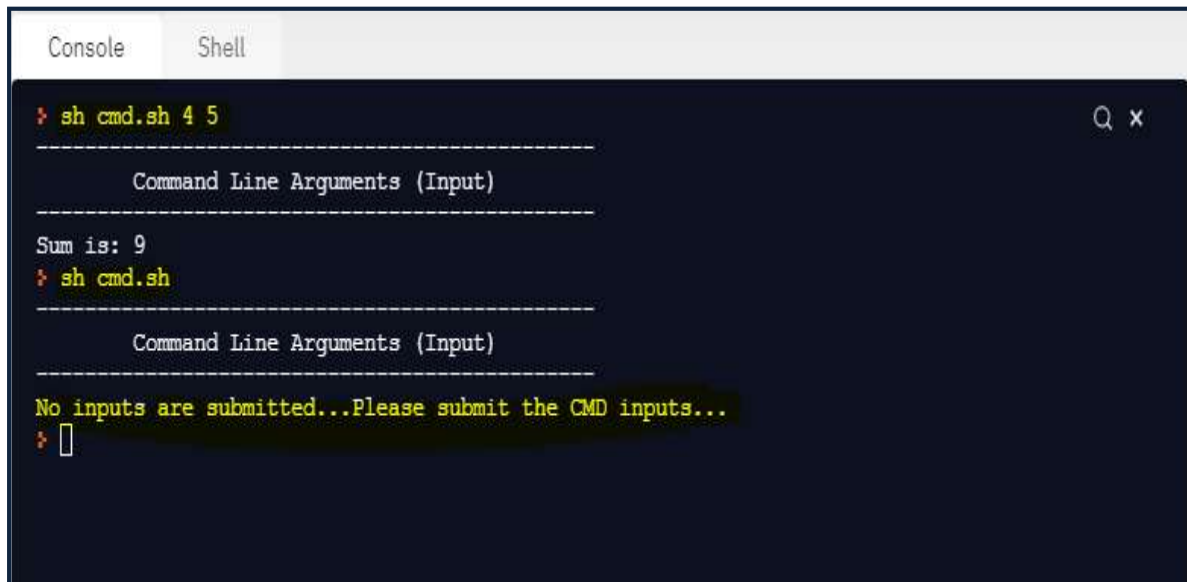
I. EXAMPLE OF COMMAND LINE INPUTS

SOURCE CODE

```
echo "-----"
echo "\t\tCommand Line Arguments (Input)"
echo "-----"
a=$1
b=$2
# check whether the command line arguments are submitted or not
if [ $# -ne 0 ]
then
    r=`expr $a + $b`
    echo "Sum is: $r"
else
    echo "No inputs are submitted...Please submit the CMD inputs..."
fi
```

This is equivalent to `if [ $# != 0 ]`

OUTPUT



```
sh cmd.sh 4 5
-----
Command Line Arguments (Input)
-----
Sum is: 9
sh cmd.sh
-----
Command Line Arguments (Input)
-----
No inputs are submitted...Please submit the CMD inputs...
[]
```

II. EXAMPLE OF COMMAND LINE INPUTS – MUCH ARGUMENTS

SOURCE CODE

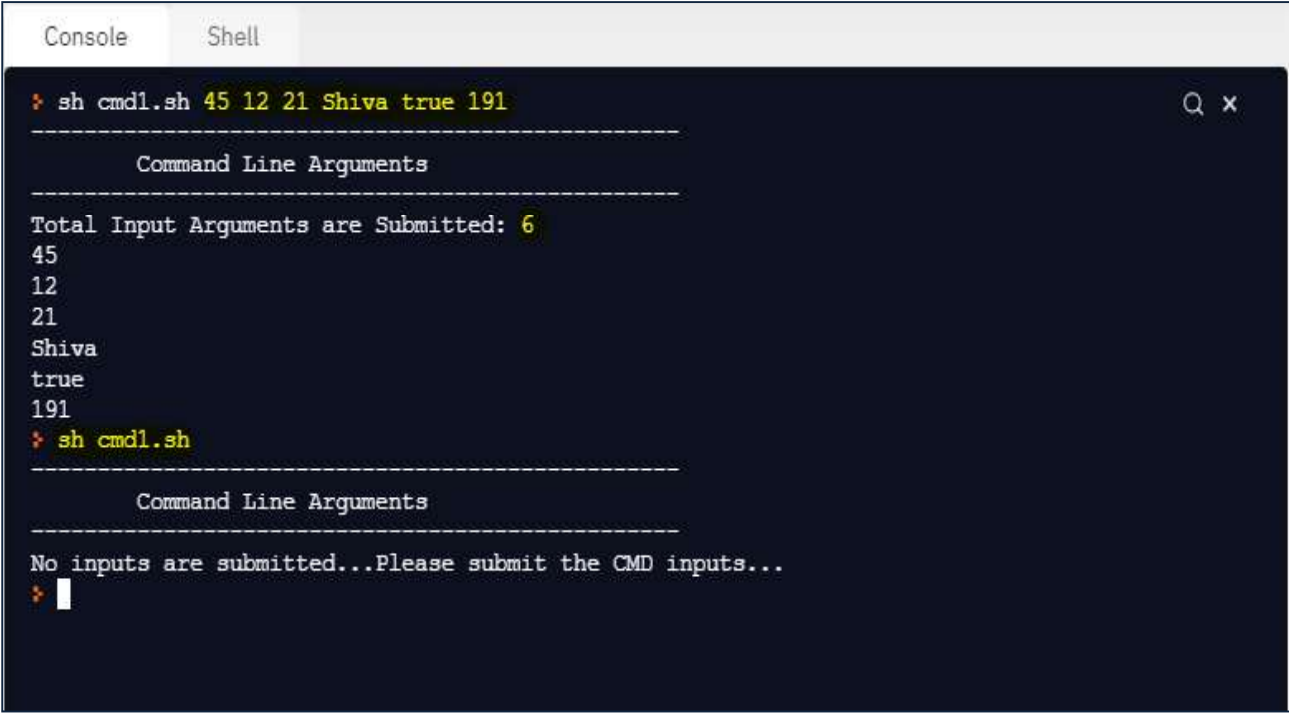
```
echo "-----"
echo "\t\tCommand Line Arguments"
echo "-----"
# check whether the command line arguments are submitted or not
if [ $# -ne 0 ]
then
    echo "Total Input Arguments are Submitted: $#"
```



This is equivalent to `if [ $# != 0 ]`

```
    for i in $*
    do
        echo $i
    done
else
    echo "No inputs are submitted...Please submit the CMD inputs..."
fi
```

OUTPUT



```
Console  Shell

❖ sh cmd1.sh 45 12 21 Shiva true 191

-----
Command Line Arguments
-----
Total Input Arguments are Submitted: 6
45
12
21
Shiva
true
191
❖ sh cmd1.sh

-----
Command Line Arguments
-----
No inputs are submitted...Please submit the CMD inputs...
❖
```

RESULT

Thus the basics of shell programming was executed successfully.